Invisibook Protocol

隐私保护跨链订单簿协议

Privacy-Preserving Cross-Chain Order Book Protocol

技术白皮书 — 协议流程详解

Version 1.0 | 2026

1 概述

Invisibook 是一种面向去中心化交易场景的隐私保护跨链订单簿协议。它解决了当前 DeFi 订单簿模型中交易数额透明化带来的一系列问题，包括前置交易（front-running）、流动性操纵（liquidity manipulation）、及其他 MEV 攻击与交易策略泄露等。

Invisibook 的核心设计理念是：报价公开、数额私密。链上订单簿仅暴露价格信息以供撮合引擎进行匹配，而订单的具体数额则以 Poseidon 哈希承诺（暂定poseidon，后续也可能考虑换成其他的hash，选型的关键是 哪个hash可以 在MPC电路 和 zk证明 性能 综合表现最优）的形式存储在链上，任何第三方无法获知订单的真实大小。当两笔订单被撮合成功后，买卖双方通过链下点对点的隐私结算协议完成交割，整个过程中双方的订单数额始终对外保密。

协议的安全性建立在三大密码学原语之上：**Poseidon** 哈希函数用于生成高效的算术友好型承诺，零知识证明（**ZK Proof**）用于在不泄露私密信息的前提下验证各类约束条件，以及**emp-ag2pc**（恶意安全两方计算）用于在两方之间执行安全的数值比较。

协议整体分为以下四个阶段：

- 阶段一：跨链充值 — 用户将资产从外部链（如 Ethereum、Solana 等 L1/L2）存入对应的跨链桥合约，在 Invisibook 链上以密文形式记入账户余额（UTXO形式），并提交 ZK Proof 证明资金所有权。
- 阶段二：私密挂单 — 用户创建订单，报价以明文公开，数额以 Poseidon 承诺隐藏，并附带余额充足性的 ZK Proof。
- 阶段三：链上透明撮合 — 撮合引擎根据明文报价执行价格优先匹配算法，将买卖双方配对。
- 阶段四：链下隐私结算 — 被匹配的双方通过 emp-ag2pc 协议安全验证承诺并比较订单大小，由小订单方将明文发送给大订单方，然后完成差额计算与状态更新。

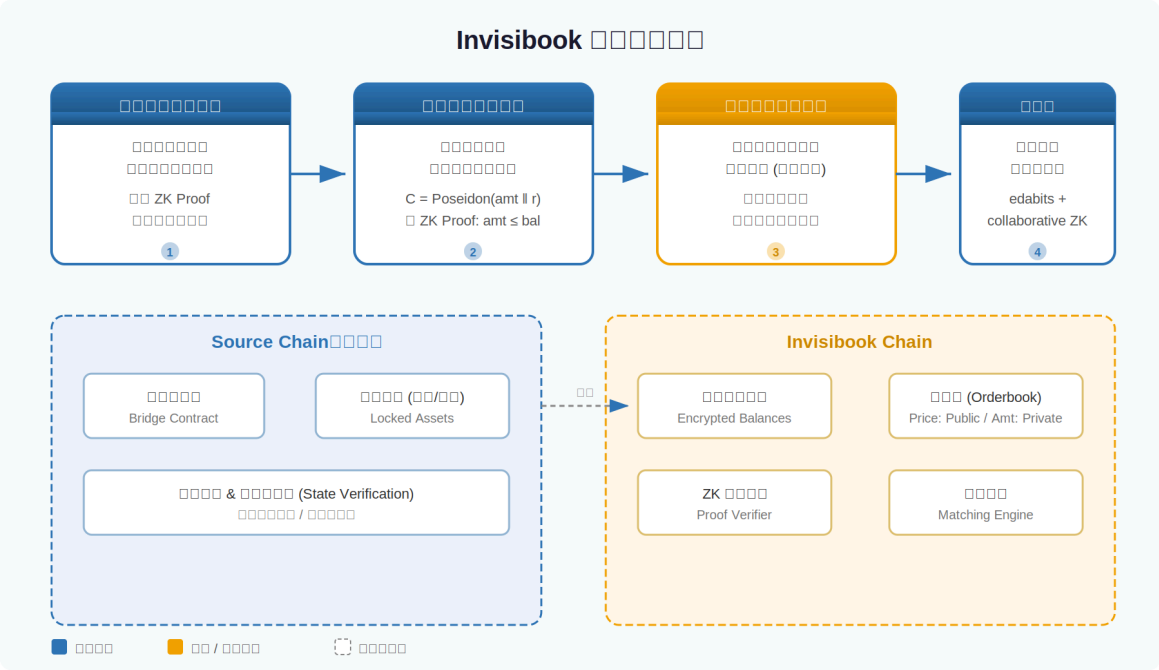

图 *1*：*Invisibook* 协议总体流程与系统架构

2 符号与基本定义

为保持后续描述的一致性，本节定义协议中使用的核心符号。

符号	类型	说明
`addr_i`	地址	用户 i 在 Invisibook 链上的账户地址
`bal_i`	密文	用户 i 的密文余额（链上存储形式）
`amt`	uint64	订单的实际数额明文（仅用户本人知晓）
`r`	随机数	256-bit 随机致盲因子
`C`	承诺值	订单数额的 Poseidon 哈希承诺
`price`	uint64	订单报价（明文，链上公开可见）
π	证明	零知识证明（ZK Proof）

Poseidon 承诺的计算方式如下：

```
C = Poseidon( amt , r )
```

其中 `,` 表示拼接操作。Poseidon 是一种针对 ZK 电路高度优化的哈希函数，其在有限域上的算术运算效率远高于 SHA-256 或 Keccak-256，因而被广泛应用于 ZKP 系统中。致盲因子 `r` 确保了相同的订单数额不会产生相同的承诺值，从而防止链上观察者通过承诺碰撞推断订单信息。

3 阶段一：跨链充值与所有权证明

3.1 流程描述

用户首先在外部源链（Source Chain）上发起一笔跨链转账交易，将原生资产或代币存入 Invisibook 部署在该链上的跨链桥合约（**Bridge Contract**）。桥合约锁定用户资产后，通过跨链消息传递机制（如状态根锚定、中继器网络或轻客户端验证等）将存款信息同步到 Invisibook 链上。源链可以是任意受支持的 L1 或 L2 网络，如 Ethereum、Solana、Arbitrum 等。

在 Invisibook 链上，用户的账户余额始终以密文形式存储。具体而言，链上状态中记录的并非用户的真实余额数值，而是经过密码学处理后的密文表示。这一设计确保了即使链上数据对所有验证者公开，任何第三方也无法直接读取用户的真实余额。

用户在完成充值后，需要在 Invisibook 链上提交一个零知识证明 **π_deposit**，证明自己确实是源链上该笔存款交易的发起者和资金的合法持有者，且链上密文余额已被正确更新。

3.2 代数描述

假设用户 Alice 向桥合约存入金额 d，其在 Invisibook 链上的旧余额密文为 bal_old，新余额密文为 bal_new，则需满足：

```
π_deposit: {
bal_new = Encrypt(Decrypt(bal_old) + d)
∧  deposit_tx ∈ source_chain_tx_root
∧  sender(deposit_tx) = addr_Alice
}
```

其中第一项约束保证了余额更新的正确性，第二项约束保证了存款交易确实被源链区块链确认，第三项约束保证了交易发起者与 Invisibook 账户的绑定关系。

4 阶段二：私密订单提交

4.1 流程描述

当用户希望在 Invisibook 的订单簿上挂出一笔订单时，需要构造一个私密订单（**Private Order**）。该订单包含两个部分：

- 明文报价 **(price)**：订单的单价信息以明文形式公开提交到链上，供撮合引擎使用。这是订单簿进行价格发现和匹配的基础。
- 数额承诺 **(C)**：订单的实际交易数量以 Poseidon 承诺的形式提交到链上。任何链上观察者只能看到一个不可逆的哈希值，无法还原出真实的订单数额。

同时，用户需要生成并提交一个零知识证明 π_order，该证明需满足以下约束：

- 承诺格式正确性：`C = Poseidon(amt , r)`
- 余额充足性：`amt × price ≤ Decrypt(bal_user)`
- 数额非负性：`amt > 0`

4.2 代数描述

$$\text{Order} = \{ \text{price}, C, \text{side}, \pi_\text{order} \}$$

其中 side ∈ {BUY, SELL} 表示订单方向。ZK Proof 的完整陈述如下：

$$\begin{gathered}
\pi_\text{order}: \{ \\
\exists\ \text{amt}, r \ \ \text{s.t.} \\
C = \text{Poseidon}(\text{amt}, r) \\
\wedge\ \ \text{amt} > 0 \\
\wedge\ \ \text{amt} \times \text{price} \leq \text{Decrypt}(\text{bal_user}) \\
\}
\end{gathered}$$

此证明确保了用户不可能挂出超出自身余额的虚假订单，同时也不可能提交非法的零数额或负数额订单。由于证明在链上被验证，任何不满足上述约束的订单都将被拒绝。

4.3 安全性分析

通过将数额隐藏在 Poseidon 承诺中，Invisibook 实现了订单数额的隐私性。链上的撮合引擎、验证者、以及其他交易者只能看到订单的价格和方向，无法得知订单的具体数量。这一设计有效抵御了以下攻击向量：

- 前置交易（**Front-running**）：攻击者无法通过观察大额订单来抢先交易。

- 流动性操纵（**Liquidity Manipulation**）：攻击者看到大额市价单时，可能抢先撤除低价位挂单并在更高价位重新挂出，迫使受害者在更差的价格成交。由于数额不可见，攻击者无法判断订单规模，也就无法判断操纵是否有利可图。
- 策略推断：做市商或竞争对手无法通过订单数量推断用户的交易策略和仓位信息。

5 阶段三：链上订单撮合

5.1 撮合机制

Invisibook 链上的撮合引擎基于价格优先（**Price Priority**）的核心原则运作。由于所有订单的报价均以明文公开，撮合引擎可以正常执行标准的订单匹配逻辑。

具体而言，撮合引擎维护一个标准的双向订单簿：

- 买单簿（**Bid Book**）：按价格从高到低排列。
- 卖单簿（**Ask Book**）：按价格从低到高排列。

当一个新的买单报价 ≥ 当前最优卖单报价时（或反之），撮合引擎将这两笔订单标记为已匹配（matched），并触发后续的链下结算流程。

5.2 撮合优先级规则

当多笔订单的报价处于可成交区间时，撮合引擎按照以下三级优先级依次排序，确定撮合的先后顺序：

- 第一优先级 — 价格最优（**Price Priority**）：买单中报价最高者优先成交，卖单中报价最低者优先成交。这是所有订单簿的基本原则，确保市场的价格发现功能正常运作。
- 第二优先级 — 区块高度最早（**Block Height Priority**）：当多笔订单报价相同时，以订单被打包进入的区块高度作为时间排序依据。区块号更低（即更早上链）的订单享有更高的撮合优先级。这一规则保证了先到先得的公平性，避免后来者在相同价格上挤占先行者的成交机会。
- 第三优先级 — **Gas Fee** 最高（**Gas Fee Priority**）：若多笔订单报价相同且处于同一区块内（即区块高度相同），则按照订单交易所支付的 gas fee 从高到低排序。支付更高 gas fee 的订单将被优先撮合。这一规则在区块内部提供了一种市场化的排序机制——当用户希望在同一价格水平上获得更高的成交优先级时，可以通过提高 gas fee 来表达这一意愿。

上述优先级规则可形式化表示为：

```
Priority(order) = ( price,  block_height↑,  gas_fee↓ )
```

即：首先按价格排序（买单降序、卖单升序），价格相同时按区块高度升序排列（越早越优先），区块高度也相同时按 gas fee 降序排列（越高越优先）。

5.3 重要设计权衡

由于订单数额是密文形式，撮合引擎在匹配阶段无法知晓两笔被匹配订单的真实数量是否相等。因此，Invisibook 的撮合结果只是一个「价格匹配」，而非传统意义上的「完全成交匹配」。真正的数量对比和差额处理需要在阶段四的链下结算中完成。

这一设计是 Invisibook 与传统订单簿撮合引擎最本质的区别：传统引擎在撮合时即可完成精确的数量匹配与部分成交处理，而 Invisibook 将数量相关的计算推迟到了链下隐私结算环节。

6 阶段四：链下隐私结算

阶段四是 Invisibook 协议中最复杂也最关键的环节。当撮合引擎将两笔订单配对后，订单的持有者（以下简称 Alice 和 Bob）需要通过链下的点对点隐私协议完成结算。

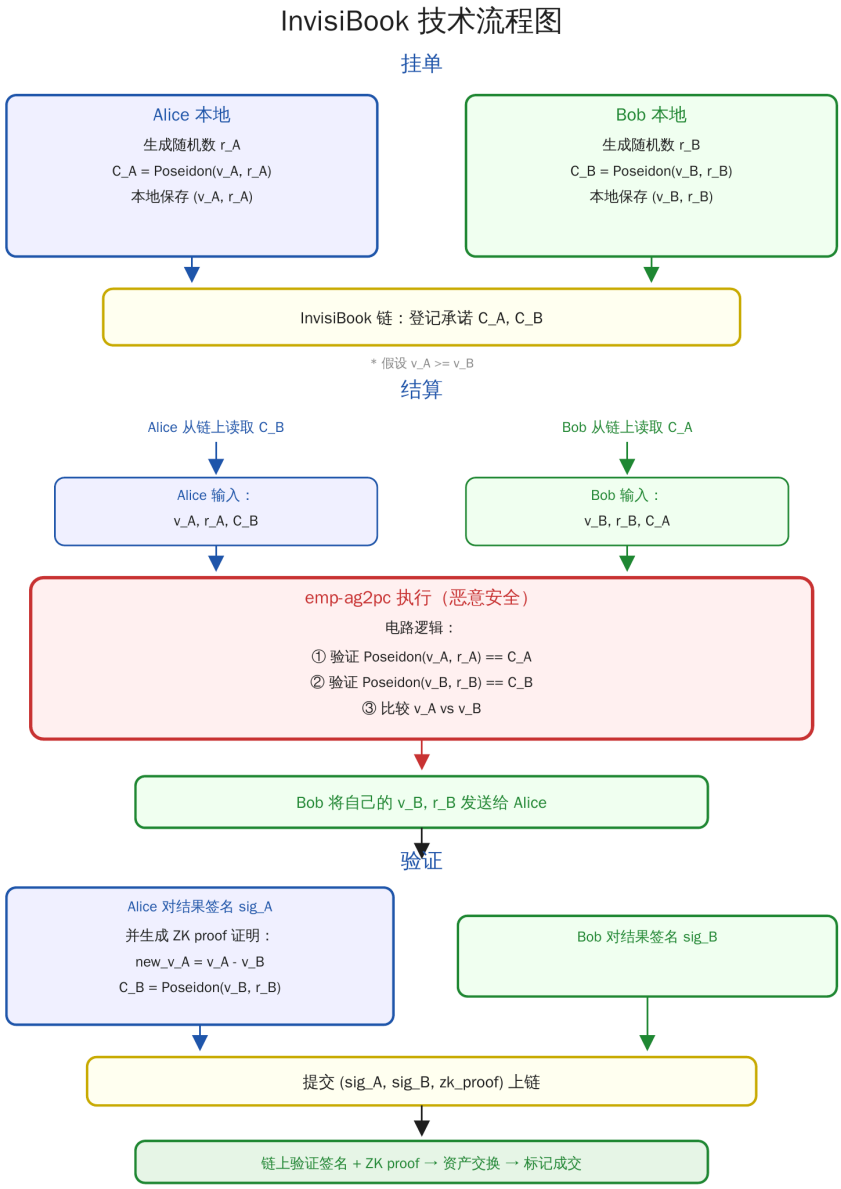

图 2：阶段四 — 链下隐私结算详细流程

6.1 安全数额比较：emp-ag2pc（恶意安全 2PC）

结算的第一步是确定哪一方的订单数额更小。但由于双方的订单数额都是私密的，任何一方都不能直接将自己的数额明文发送给对方（否则在比较阶段即泄露了隐私）。

Invisibook 采用 **emp-ag2pc** 框架 进行恶意安全的 两方安全计算（暂定**emp-ag2pc**，但核心是恶意安全的**MPC**，后续可能换成其他的框架

），在不泄露任何一方输入的前提下完成承诺验证与数值比较。

emp-ag2pc（**Authenticated Garbled Circuits for 2PC**）是一种基于混淆电路（Garbled Circuits）的恶意安全两方计算框架。在 Invisibook 的结算场景中，Alice 和 Bob 各自将自己的秘密输入（订单数额 amt 和致盲因子 r）提供给 2PC 电路，电路在不泄露任何一方明文的前提下执行以下三步逻辑：① 验证 Poseidon(amt_A, r_A) == C_A；② 验证 Poseidon(amt_B, r_B) == C_B；③ 比较 amt_A 与 amt_B 的大小。电路仅输出比较结果（即哪一方为小订单方），不揭示任何数额明文。

承诺验证与比较的电路设计值得注意的是，承诺验证被纳入 2PC 电路内部执行，这意味着双方无需在比较前先各自公开自己的承诺打开（opening）。电路内部直接以秘密共享的方式计算 Poseidon 哈希并与链上承诺比对，从而将承诺验证与数值比较合并为一次原子操作，避免了任何中间信息泄露。比较完成后，小订单方再通过点对点加密通道将自己的明文数额和致盲因子发送给大订单方（此步骤在 2PC 电路之外执行）。

emp-ag2pc 电路的输出是一个比特值 b ∈ {0, 1}，表示比较结果

6.2 小订单方结算（以 Alice 为例）

假设比较结果为 amt_A <= amt_B，即 Alice 持有较小的订单。此时 Alice 需要执行以下操作：

6.2.1 点对点发送明文

Alice 通过安全的点对点通信，将自己的订单数额明文 `amt_A` 和对应的 256-bit 随机致盲因子 `r_A` 发送给 Bob。此时 Bob 可以在本地验证：

```
Poseidon(amt_A , r_A) == C_A
```

其中 `C_A` 是 Alice 之前在链上发布的订单承诺。如果验证通过，Bob 可以确信 Alice 发送的明文数据与链上承诺一致，不存在欺骗。

6.2.2 更新链上状态

Alice 将以下更新提交到链上：

- 将自己的订单状态标记为「已完全成交」（fully filled）
- 更新自己的密文账户余额（扣除已成交的 amt_A 部分，计入对应的买入/卖出资产 ）

此证明来自步骤 6.1 中双方协作生成的比较证明。

6.3 大订单方结算（以 Bob 为例）

Bob 收到 Alice 发来的 amt_A 和 r_A 后，首先在本地验证承诺的一致性。验证通过后，Bob 在本地计算新的订单剩余数量：

```
amt_B' = amt_B - amt_A
```

随后 Bob 选取一个新的随机致盲因子 r_B'，计算新的订单承诺：

```
C_B' = Poseidon(amt_B' , r_B')
```

Bob 将以下更新提交到链上：

- 将自己的订单承诺从 C_B 更新为 C_B'（剩余订单继续挂单 ）
- 更新自己的密文账户余额（计入本次已结算的 amt_A 部分）
- 附上零知识证明 π_B，需同时证明以下两个陈述：

```
π_B: {
(1)  amt_B' = amt_B - amt_A
(2)  Poseidon(amt_A , r_A) = C_A
}
```

其中第 (1) 项证明了新订单数额是旧订单数额减去小订单数额的正确结果（确保 Bob 没有篡改计算）。第 (2) 项证明了 Bob 所使用的 amt_A 确实与链上之前已承诺的 Alice 的订单密文 C_A 一致（防止 Bob 伪造小订单数量来获取不正当利益）。

6.4 举证

如果Alice作为小订单持有方，当他得知自己是小订单之后可能会选择不给Bob发送amt_A和r_A。此时，当超过5个区块时间之后，Bob可以向链上发起强制申请，并附带自己的一个公钥Pub_B，强制要求Alice将自己的amt_A 和r_A 用 pub_B加密（amt_A, r_A）并发到链上，当Alice看到强制申请之后，需在10个区块时间内发送 pub_B(amt_A, r_A) 到链上，否则Alice的该订单将会被直接冻结72小时，而Bob的订单可以重新与其他加密订单进行撮合。

Bob在看到链上出现pub_B(amt_A, r_A）之后下载到本地，然后解密，并按照6.1的步骤进行验证和结算。

7 各阶段证明义务汇总

下表汇总了协议各阶段中涉及的零知识证明及其所证明的核心陈述：

阶段	证明	核心陈述
阶段一	π_deposit	余额更新正确 ∧ 存款交易存在于源链 ∧ 发起者身份绑定
阶段二	π_order	C = Poseidon(amt , r) ∧ amt > 0 ∧ amt × price ≤ bal
阶段四 (比较)	π_cmp	b = (amt_A < amt_B) 是双方数额的正确比较结果
阶段四 (小单方)	π_A	amt_A < amt_B（确认自身为小订单方）
阶段四 (大单方)	π_B	amt_B' = amt_B - amt_A ∧ Poseidon(amt_A , r_A) = C_A

8 安全性质总结

Invisibook 协议在不同层面提供了以下安全保证：

安全性质	实现机制	说明
订单数额隐私	Poseidon 承诺 + ZKP	链上仅存储承诺值，任何第三方无法还原真实数额
账户余额隐私	密文余额 + ZKP	用户余额始终以密文存储，余额更新附带正确性证明
安全比较	emp-ag2pc（恶意安全 2PC）	双方可比较订单大小而不互相泄露明文数额
结算正确性	链上 ZKP 验证	差额计算、承诺一致性均通过零知识证明在链上强制验证
抗 **Front-running**	数额不可见	攻击者无法通过观察订单数额来实施抢先交易或流动性操纵等 MEV 攻击
跨链安全性	源链状态根验证 + 存款证明	充值操作通过源链状态根验证，确保跨链资产映射的正确性